MediCare AI - Healthcare Chatbot

Logic Documentation

AI Engineer Technical Assignment
OpenRouter + LangChain + FAISS + Streamlit

1. Overview

This document explains the internal logic, prompt engineering strategy, safety measures, and design assumptions of the Healthcare AI Chatbot. The chatbot uses Retrieval-Augmented Generation (RAG), conversation memory, and prompt engineering to provide safe, accurate, and empathetic health information.

2. Query Processing Pipeline

2.1 Query Classification

When a user sends a message, it first passes through a Query Classifier that categorizes it into one of four types:

Category	Trigger	Handler
emergency	Keywords: chest pain, can't breathe, suicide	Immediate emergency response
greeting	Short messages matching greeting patterns	Welcome message with features
out_of_scope	Non-health topics (weather, code, shopping)	Redirect to health topics
health_query	All other health-related questions	Full RAG + LLM pipeline

Implementation Details:

- Emergency detection uses a curated keyword list covering cardiac events, breathing emergencies, stroke symptoms, severe bleeding, and mental health crises
- Greeting detection uses regex patterns with a word-count threshold (6 words or less) to avoid false positives
- Out-of-scope detection checks for strong non-health indicators like weather, stock price, programming

2.2 RAG Retrieval

For health queries, the system retrieves relevant context from the FAISS vector store:

1. Query Embedding: The user question is embedded using all-MiniLM-L6-v2 (384-dimensional)
2. Similarity Search: FAISS performs cosine similarity search against the knowledge base
3. Top-K Selection: The 3 most relevant documents are retrieved
4. Context Formatting: Retrieved documents are formatted with source metadata

2.3 Response Generation

The LLM generates a response using the system prompt with role definition and safety guidelines, retrieved RAG context from the knowledge base, recent conversation history (last 3 exchanges), and the user current question.

2.4 Post-Processing

After generation, the medical disclaimer is checked and added if missing, source citations are appended from retrieved documents, and the response is stored in conversation memory.

3. Prompt Engineering Strategy

3.1 System Prompt Architecture

The system prompt follows a layered structure:

Layer	Content	Purpose
1: Role Definition	Empathetic Healthcare AI Assistant	Sets persona and expertise
2: Scope Boundaries	CAN do vs MUST NOT do lists	Prevents harmful outputs
3: Response Style	Empathetic tone, clear language	Guides response quality
4: Safety Protocol	Emergency handling, disclaimers	Ensures user safety
5: RAG Integration	How to use context, cite sources	Grounds in facts

3.2 Key Techniques

Technique	Application	Purpose
Role Prompting	Healthcare AI persona	Sets expertise level
Constraint Setting	No diagnoses/prescriptions	Prevents harm
Context Injection	RAG docs in system prompt	Grounds in facts
Guardrail Prompting	Safety rules in every prompt	Consistent safety
Response Templates	Emergency/greeting/OOS	Consistent format
Chain-of-Thought	Classification then Generation	Structured reasoning

4. Conversation Memory

4.1 Dual Memory Architecture

LangChain-style buffer memory for LLM context maintains last 10 exchanges formatted as Human/AI message pairs, injected into the LLM prompt.

Application-level Chat History for UI display stores up to 20 messages with timestamps and source metadata for rendering the chat interface.

4.2 Context-Aware Follow-ups

When a user asks a follow-up question (e.g., What about treatment? after asking about diabetes), the conversation memory provides previous exchanges to the LLM, which understands the context and generates a coherent response.

5. Safety Measures

5.1 Emergency Detection

Keyword-based detection covers cardiac events, breathing emergencies, stroke symptoms, severe bleeding, and mental health crises. When detected, the chatbot immediately displays emergency contact numbers and does NOT attempt medical treatment advice.

5.2 Diagnosis Prevention

The system prompt explicitly states that the AI MUST NOT provide diagnoses or recommend medications. Responses are always phrased informatively.

5.3 Medical Disclaimers

Every health response includes a disclaimer stating this is general health information, not medical advice.

5.4 Scope Enforcement

Non-health queries receive a friendly redirect listing available healthcare topics.

6. Assumptions Made

API Availability: Assumes OpenRouter API is accessible with a valid API key

English Language: Designed for English-language queries

General Health Info: Knowledge base provides general information, not personalized advice

CPU Environment: Embeddings run on CPU (no GPU required)

Single User: Designed for single-user sessions

Internet Required: Requires internet for API calls

Free Tier Usage: Designed to work within OpenRouter free API tier limits

7. Error Handling

Error Scenario	Handling
Missing API Key	Clear error message with setup instructions
API Rate Limit	Graceful error with retry suggestion
Vector Store Missing	Auto-creates from knowledge base JSON
LLM Generation Error	Safe fallback recommending professional consultation
Empty/Invalid Input	Handled by Streamlit chat input validation
Network Errors	Caught and displayed as user-friendly messages

8. Performance Considerations

Embedding Cache: FAISS index loaded once at startup and reused

Memory Window: Limited to 10 exchanges to control token usage

Response Streaming: Simulated typing effect for better UX

Lazy Initialization: Components initialized only when needed

Chunk Size: 500 chars balances context quality with retrieval accuracy